

Master

April 12, 2026

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
import geopandas as gpd
import plotly.graph_objects as go
from shapely.geometry import Point
from pyproj import Transformer
```

```
[2]: BASE_DIR = Path.cwd()

PARQUET_FILES = {
    "base_map": BASE_DIR / "alaska_base_map.parquet",
    "extent": BASE_DIR / "north_slope_extent.parquet",
    "assessment_units": BASE_DIR / "north_slope_assessment_units.parquet",
    "seismic_2d": BASE_DIR / "clean_2d_seismic.parquet",
    "seismic_3d_inventory": BASE_DIR / "clean_3d_seismic.parquet",
    "wells": BASE_DIR / "clean_well_locations.parquet",
}

XYZ_DIR = BASE_DIR / "raw_data" / "north_slope_depth_grids"

TARGET_ANALYSIS_CRS = "EPSG:3338"
TARGET_WEB_CRS = "EPSG:4326"

SURFACE_DECIMATE = 8    # increase for lighter files, reduce for more detail
LINE_VERTEX_STEP = 1    # keep every vertex for now
```

```
[3]: def load_gdf(fp, fallback_crs=None):
    gdf = gpd.read_parquet(fp).copy()
    if "geometry" not in gdf.columns:
        raise ValueError(f"{fp.name} has no geometry column")
    gdf = gpd.GeoDataFrame(gdf, geometry="geometry")
    if gdf.crs is None and fallback_crs is not None:
        gdf = gdf.set_crs(fallback_crs, allow_override=True)
    return gdf

def ensure_crs(gdf, target_crs):
    if gdf.crs is None:
```

```

        raise ValueError("Layer has no CRS")
    if str(gdf.crs) != target_crs:
        return gdf.to_crs(target_crs)
    return gdf.copy()

```

```

[4]: layers = {}

for name, fp in PARQUET_FILES.items():
    if fp.exists():
        layers[name] = load_gdf(fp)
        print(f"Loaded {name}: {len(layers[name])} rows | CRS={layers[name].
↳crs}")
    else:
        print(f"Missing: {fp.name}")

```

```

Loaded base_map: 9 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":
"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",
"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]}, "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]},
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]}, "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":
71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}
Loaded extent: 1 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":
"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",

```

```

"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]], "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]],
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]], "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":
71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}
Loaded assessment_units: 6 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":
"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",
"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]], "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]],
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]], "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":
71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}
Loaded seismic_2d: 26 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":

```

```

"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",
"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]}, "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]},
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]}, "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":
71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}
Loaded seismic_3d_inventory: 36 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":
"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",
"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]}, "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]},
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]}, "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":

```

```

71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}
Loaded wells: 10250 rows | CRS={"$schema":
"https://proj.org/schemas/v0.7/projjson.schema.json", "type": "ProjectedCRS",
"name": "NAD83 / Alaska Albers", "base_crs": {"name": "NAD83", "datum": {"type":
"GeodeticReferenceFrame", "name": "North American Datum 1983", "ellipsoid":
{"name": "GRS 1980", "semi_major_axis": 6378137, "inverse_flattening":
298.257222101}}, "coordinate_system": {"subtype": "ellipsoidal", "axis":
[{"name": "Geodetic latitude", "abbreviation": "Lat", "direction": "north",
"unit": "degree"}, {"name": "Geodetic longitude", "abbreviation": "Lon",
"direction": "east", "unit": "degree"}]}, "id": {"authority": "EPSG", "code":
4269}}, "conversion": {"name": "Alaska Albers (meter)", "method": {"name":
"Albers Equal Area", "id": {"authority": "EPSG", "code": 9822}}, "parameters":
[{"name": "Latitude of false origin", "value": 50, "unit": "degree", "id":
{"authority": "EPSG", "code": 8821}}, {"name": "Longitude of false origin",
"value": -154, "unit": "degree", "id": {"authority": "EPSG", "code": 8822}},
{"name": "Latitude of 1st standard parallel", "value": 55, "unit": "degree",
"id": {"authority": "EPSG", "code": 8823}}, {"name": "Latitude of 2nd standard
parallel", "value": 65, "unit": "degree", "id": {"authority": "EPSG", "code":
8824}}, {"name": "Easting at false origin", "value": 0, "unit": "metre", "id":
{"authority": "EPSG", "code": 8826}}, {"name": "Northing at false origin",
"value": 0, "unit": "metre", "id": {"authority": "EPSG", "code": 8827}}]},
"coordinate_system": {"subtype": "Cartesian", "axis": [{"name": "Easting",
"abbreviation": "X", "direction": "east", "unit": "metre"}, {"name": "Northing",
"abbreviation": "Y", "direction": "north", "unit": "metre"}]}, "scope":
"Topographic mapping (small scale).", "area": "United States (USA) - Alaska.",
"bbox": {"south_latitude": 51.3, "west_longitude": 172.42, "north_latitude":
71.4, "east_longitude": -129.99}, "id": {"authority": "EPSG", "code": 3338}}

```

```

[5]: for name in list(layers.keys()):
      layers[name] = ensure_crs(layers[name], TARGET_ANALYSIS_CRS)

for name, gdf in layers.items():
    print(name, gdf.crs)

```

```

base_map EPSG:3338
extent EPSG:3338
assessment_units EPSG:3338
seismic_2d EPSG:3338
seismic_3d_inventory EPSG:3338
wells EPSG:3338

```

```

[56]: import pandas as pd
      from shapely.geometry.base import BaseGeometry

      def inspect_layer_geometries(gdf, layer_name):
          print(f"\n--- {layer_name} ---")
          print("rows:", len(gdf))
          print("crs:", gdf.crs)

```

```

type_counts = gdf["geometry"].apply(lambda x: type(x).__name__).
↳value_counts(dropna=False)
print("\npython types in geometry column:")
print(type_counts)

null_mask = gdf["geometry"].isna()
print("\nnull geometry rows:", null_mask.sum())

non_geom_mask = ~gdf["geometry"].apply(lambda x: isinstance(x,
↳BaseGeometry) or pd.isna(x))
print("non-geometry rows:", non_geom_mask.sum())

if non_geom_mask.sum() > 0:
    display(gdf.loc[non_geom_mask].head(10))

geom_mask = gdf["geometry"].apply(lambda x: isinstance(x, BaseGeometry))
geom_only = gdf.loc[geom_mask].copy()

if len(geom_only) > 0:
    print("\ngeometry types:")
    print(geom_only.geometry.geom_type.value_counts(dropna=False))
    print("\nempty geometries:", geom_only.geometry.is_empty.sum())
    print("invalid geometries:", (~geom_only.geometry.is_valid).sum())
else:
    print("\nNo valid shapely geometries found.")

```

```

[57]: for name, gdf in layers.items():
    inspect_layer_geometries(gdf, name)

```

```
--- base_map ---
```

```
rows: 9
```

```
crs: EPSG:3338
```

```
python types in geometry column:
```

```
geometry
```

```
Polygon    9
```

```
Name: count, dtype: int64
```

```
null geometry rows: 0
```

```
non-geometry rows: 0
```

```
geometry types:
```

```
Polygon    9
```

```
Name: count, dtype: int64
```

```
empty geometries: 0
```

```

invalid geometries: 0

--- extent ---
rows: 1
crs: EPSG:3338

python types in geometry column:
geometry
Polygon      1
Name: count, dtype: int64

null geometry rows: 0
non-geometry rows: 0

geometry types:
Polygon      1
Name: count, dtype: int64

empty geometries: 0
invalid geometries: 0

--- assessment_units ---
rows: 6
crs: EPSG:3338

python types in geometry column:
geometry
Polygon      5
MultiPolygon  1
Name: count, dtype: int64

null geometry rows: 0
non-geometry rows: 0

geometry types:
Polygon      5
MultiPolygon  1
Name: count, dtype: int64

empty geometries: 0
invalid geometries: 0

--- seismic_2d ---
rows: 26
crs: EPSG:3338

python types in geometry column:
geometry

```

```
MultiLineString    23
LineString          3
Name: count, dtype: int64
```

```
null geometry rows: 0
non-geometry rows: 0
```

```
geometry types:
MultiLineString    23
LineString          3
Name: count, dtype: int64
```

```
empty geometries: 0
invalid geometries: 0
```

```
--- seismic_3d_inventory ---
rows: 36
crs: EPSG:3338
```

```
python types in geometry column:
geometry
Polygon          21
MultiPolygon      15
Name: count, dtype: int64
```

```
null geometry rows: 0
non-geometry rows: 0
```

```
geometry types:
Polygon           21
MultiPolygon       15
Name: count, dtype: int64
```

```
empty geometries: 0
invalid geometries: 6
```

```
--- wells ---
rows: 10250
crs: EPSG:3338
```

```
python types in geometry column:
geometry
Point            9894
NoneType          356
Name: count, dtype: int64
```

```
null geometry rows: 356
non-geometry rows: 0
```

```
geometry types:
Point      9894
Name: count, dtype: int64
```

```
empty geometries: 0
invalid geometries: 0
```

```
[7]: from shapely.geometry.base import BaseGeometry

def clean_layer_geometry(gdf, layer_name):
    gdf = gdf.copy()

    # keep only actual shapely geometries
    gdf = gdf[gdf["geometry"].apply(lambda x: isinstance(x, BaseGeometry))].
    ↪copy()

    # drop empty geometries
    gdf = gdf[~gdf.geometry.is_empty].copy()

    # optionally drop invalid geometries
    gdf = gdf[gdf.geometry.is_valid].copy()

    print(f"{layer_name}: cleaned to {len(gdf)} rows")
    return gdf

for name in list(layers.keys()):
    layers[name] = clean_layer_geometry(layers[name], name)
```

```
base_map: cleaned to 9 rows
extent: cleaned to 1 rows
assessment_units: cleaned to 6 rows
seismic_2d: cleaned to 26 rows
seismic_3d_inventory: cleaned to 30 rows
wells: cleaned to 9894 rows
```

```
[8]: def geometry_to_vertices(gdf, layer_name, layer_group, default_depth=0.0):
    rows = []

    for idx, row in gdf.iterrows():
        geom = row.geometry
        if geom is None or geom.is_empty:
            continue

        attrs = row.drop(labels=["geometry"]).to_dict()

        if geom.geom_type == "Point":
            coords = [geom.coords[0]]
```

```

elif geom.geom_type == "MultiPoint":
    coords = []
    for part in geom.geoms:
        coords.extend(list(part.coords))

elif geom.geom_type == "LineString":
    coords = list(geom.coords)[:LINE_VERTEX_STEP]

elif geom.geom_type == "MultiLineString":
    coords = []
    for part in geom.geoms:
        coords.extend(list(part.coords)[:LINE_VERTEX_STEP])

elif geom.geom_type == "Polygon":
    coords = list(geom.exterior.coords)[:LINE_VERTEX_STEP]

elif geom.geom_type == "MultiPolygon":
    coords = []
    for part in geom.geoms:
        coords.extend(list(part.exterior.coords)[:LINE_VERTEX_STEP])

else:
    continue

for order_id, (x, y, *rest) in enumerate(coords):
    rows.append({
        "layer_group": layer_group,
        "layer_name": layer_name,
        "feature_id": idx,
        "vertex_order": order_id,
        "feature_type": geom.geom_type,
        "x_3338": x,
        "y_3338": y,
        "depth_m": default_depth,
        **attrs
    })

out = pd.DataFrame(rows)

if len(out) == 0:
    return out

geom_points = gpd.GeoSeries(
    [Point(xy) for xy in zip(out["x_3338"], out["y_3338"])],
    crs=TARGET_ANALYSIS_CRS
).to_crs(TARGET_WEB_CRS)

```

```

out["lon"] = geom_points.x.values
out["lat"] = geom_points.y.values
return out

```

```

[9]: master_2d_parts = []

layer_specs = [
    ("seismic_2d", "seismic", 0.0),
    ("seismic_3d_inventory", "seismic_inventory", 0.0),
    ("wells", "wells", 0.0),
    ("assessment_units", "geology_framework", 0.0),
    ("extent", "extent", 0.0),
]

for layer_name, layer_group, z0 in layer_specs:
    if layer_name in layers:
        part = geometry_to_vertices(
            layers[layer_name],
            layer_name=layer_name,
            layer_group=layer_group,
            default_depth=z0
        )
        part["source_file"] = PARQUET_FILES[layer_name].name
        master_2d_parts.append(part)
        print(layer_name, part.shape)

master_2d = pd.concat(master_2d_parts, ignore_index=True)
master_2d.head()

```

```

seismic_2d (349302, 17)
seismic_3d_inventory (106213, 18)
wells (9894, 14)
assessment_units (73463, 16)
extent (5, 12)

```

```

[9]: layer_group layer_name feature_id vertex_order feature_type \
0 seismic seismic_2d 0 0 MultiLineString
1 seismic seismic_2d 0 1 MultiLineString
2 seismic seismic_2d 0 2 MultiLineString
3 seismic seismic_2d 0 3 MultiLineString
4 seismic seismic_2d 0 4 MultiLineString

x_3338 y_3338 depth_m \
0 236731.9887 2.285481e+06 0.0
1 236332.2632 2.284195e+06 0.0
2 236731.9887 2.285481e+06 0.0
3 253775.7330 2.279892e+06 0.0

```

```
4 234675.5291 2.285489e+06 0.0
```

	survey_name	year_acquired	...	\
0	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
1	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
2	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
3	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
4	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	

	source_file	square_miles	survey_type	operator	field	region	\
0	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
1	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
2	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
3	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
4	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	

	province	tps_name	au_name	name
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

```
[5 rows x 26 columns]
```

```
[10]: def load_xyz_points(file_path):
        rows = []
        with open(file_path, "r") as f:
            for line in f.readlines()[8:]:
                parts = line.strip().split()
                if len(parts) == 5:
                    rows.append(parts)

        df = pd.DataFrame(rows, columns=["x_3338", "y_3338", "z_native", "lon", "lat"])
        df = df.apply(pd.to_numeric, errors="coerce").dropna()

        return df
```

```
[11]: surface_parts = []

for fp in sorted(XYZ_DIR.glob("*.XYZ")):
    df = load_xyz_points(fp)

    # decimate so files do not get huge
    df = df.iloc[:, :SURFACE_DECIMATE, :].copy()
```

```

df["surface_name"] = fp.stem
df["source_file"] = fp.name

# use z_native as depth/elevation value
df["depth_m"] = df["z_native"]

surface_parts.append(df)

print(fp.name, df.shape)

master_3d = pd.concat(surface_parts, ignore_index=True)
master_3d.head()

```

```

NSLCU-Shublik.XYZ (11310, 8)
NSLCU.XYZ (11310, 8)
NSbasement.XYZ (11310, 8)
NSshublik-basement.XYZ (11310, 8)
NSshublik.XYZ (11310, 8)
NStopo-LCU.XYZ (11310, 8)
NStopo-basement.XYZ (11310, 8)
NStopo.XYZ (11310, 8)

```

```

[11]:   x_3338   y_3338  z_native      lon      lat  surface_name \
0 -630000  2000000         0 -168.67649  67.32879  NSLCU-Shublik
1 -606000  2000000         0 -168.13443  67.37537  NSLCU-Shublik
2 -582000  2000000         0 -167.59045  67.42022  NSLCU-Shublik
3 -558000  2000000         0 -167.04462  67.46335  NSLCU-Shublik
4 -534000  2000000         0 -166.49700  67.50473  NSLCU-Shublik

```

```

      source_file  depth_m
0  NSLCU-Shublik.XYZ      0
1  NSLCU-Shublik.XYZ      0
2  NSLCU-Shublik.XYZ      0
3  NSLCU-Shublik.XYZ      0
4  NSLCU-Shublik.XYZ      0

```

```

[12]: master_2d.to_parquet("north_slope_master_2d_layers.parquet", index=False)
      master_3d.to_parquet("north_slope_master_3d_surfaces.parquet", index=False)

print("Saved:")
print(" - north_slope_master_2d_layers.parquet")
print(" - north_slope_master_3d_surfaces.parquet")

```

```

Saved:
- north_slope_master_2d_layers.parquet
- north_slope_master_3d_surfaces.parquet

```

```
[13]: print(master_2d.shape)
print(master_2d.columns.tolist())
display(master_2d.head())

print(master_3d.shape)
print(master_3d.columns.tolist())
display(master_3d.head())
```

(538877, 26)

```
['layer_group', 'layer_name', 'feature_id', 'vertex_order', 'feature_type',
'x_3338', 'y_3338', 'depth_m', 'survey_name', 'year_acquired', 'year_released',
'company', 'line_miles', 'type', 'lon', 'lat', 'source_file', 'square_miles',
'survey_type', 'operator', 'field', 'region', 'province', 'tps_name', 'au_name',
'name']
```

	layer_group	layer_name	feature_id	vertex_order	feature_type	\
0	seismic	seismic_2d	0	0	MultiLineString	
1	seismic	seismic_2d	0	1	MultiLineString	
2	seismic	seismic_2d	0	2	MultiLineString	
3	seismic	seismic_2d	0	3	MultiLineString	
4	seismic	seismic_2d	0	4	MultiLineString	

	x_3338	y_3338	depth_m	\
0	236731.9887	2.285481e+06	0.0	
1	236332.2632	2.284195e+06	0.0	
2	236731.9887	2.285481e+06	0.0	
3	253775.7330	2.279892e+06	0.0	
4	234675.5291	2.285489e+06	0.0	

	survey_name	year_acquired	...	\
0	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
1	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
2	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
3	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	
4	Pike Bottom Hole Planning 2D Reprocessing	NaN	...	

	source_file	square_miles	survey_type	operator	field	region	\
0	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
1	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
2	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
3	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	
4	clean_2d_seismic.parquet	NaN	NaN	NaN	NaN	NaN	

	province	tps_name	au_name	name
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN

```
4      NaN      NaN      NaN      NaN
```

```
[5 rows x 26 columns]
```

```
(90480, 8)
```

```
['x_3338', 'y_3338', 'z_native', 'lon', 'lat', 'surface_name', 'source_file',  
'depth_m']
```

	x_3338	y_3338	z_native	lon	lat	surface_name	\
0	-630000	2000000	0	-168.67649	67.32879	NSLCU-Shublik	
1	-606000	2000000	0	-168.13443	67.37537	NSLCU-Shublik	
2	-582000	2000000	0	-167.59045	67.42022	NSLCU-Shublik	
3	-558000	2000000	0	-167.04462	67.46335	NSLCU-Shublik	
4	-534000	2000000	0	-166.49700	67.50473	NSLCU-Shublik	

	source_file	depth_m
0	NSLCU-Shublik.XYZ	0
1	NSLCU-Shublik.XYZ	0
2	NSLCU-Shublik.XYZ	0
3	NSLCU-Shublik.XYZ	0
4	NSLCU-Shublik.XYZ	0

```
[14]: print(master_2d["layer_name"].value_counts(dropna=False))  
      print(master_3d["surface_name"].value_counts(dropna=False))
```

```
layer_name  
seismic_2d      349302  
seismic_3d_inventory  106213  
assessment_units    73463  
wells            9894  
extent           5  
Name: count, dtype: int64  
surface_name  
NSLCU-Shublik    11310  
NSLCU            11310  
NSbasement       11310  
NSshublik-basement  11310  
NSshublik        11310  
NStopo-LCU       11310  
NStopo-basement   11310  
NStopo           11310  
Name: count, dtype: int64
```

```
[15]: from pathlib import Path  
      import numpy as np  
      import pandas as pd  
      import geopandas as gpd
```

```
# -----
```

```

# settings
# -----
XYZ_DIR = Path("raw_data/north_slope_depth_grids") # adjust only if needed
TARGET_ANALYSIS_CRIS = "EPSG:3338"
TARGET_WEB_CRIS = "EPSG:4326"

# keep only the surfaces you want
XYZ_SURFACES = [
    "NStopo.XYZ",
    "NSLCU.XYZ",
    "NSshublik.XYZ",
    "NSbasement.XYZ",
    "NStopo-LCU.XYZ",
    "NSLCU-Shublik.XYZ",
    "NSshublik-basement.XYZ",
    "NStopo-basement.XYZ",
]

# -----
# same idea as your depth-grid notebook
# -----
def load_xyz_grid(file_path):
    rows = []

    with open(file_path, "r") as f:
        lines = f.readlines()

    # your notebook skipped the first 8 lines
    for line in lines[8:]:
        parts = line.strip().split()
        if len(parts) < 3:
            continue

        try:
            x = float(parts[0])
            y = float(parts[1])
            z = float(parts[2])
            rows.append((x, y, z))
        except ValueError:
            continue

    df = pd.DataFrame(rows, columns=["x_3338", "y_3338", "z_native"])

    grid = df.pivot(index="y_3338", columns="x_3338", values="z_native")
    grid = grid.sort_index(ascending=False)

    return grid

```

```

def grid_to_long_df(grid, surface_name):
    df_long = (
        grid
        .stack()
        .reset_index()
    )
    df_long.columns = ["y_3338", "x_3338", "z_native"]

    # reorder to match the rest of your project
    df_long = df_long[["x_3338", "y_3338", "z_native"]].copy()
    df_long["surface_name"] = surface_name
    df_long["source_file"] = f"{surface_name}.XYZ"

    return df_long

# -----
# load all xyz grids
# -----
grids = {}

for fname in XYZ_SURFACES:
    fp = XYZ_DIR / fname
    if fp.exists():
        surface_name = fp.stem
        grids[surface_name] = load_xyz_grid(fp)
        print(f"Loaded {surface_name}: {grids[surface_name].shape}")
    else:
        print(f"Missing: {fp}")

# -----
# convert all grids to one long dataframe
# -----
surface_parts = []

for surface_name, grid in grids.items():
    part = grid_to_long_df(grid, surface_name)
    surface_parts.append(part)

master_3d = pd.concat(surface_parts, ignore_index=True)

# -----
# add lon / lat from projected x/y
# -----
geom = gpd.points_from_xy(master_3d["x_3338"], master_3d["y_3338"])
gdf = gpd.GeoDataFrame(master_3d.copy(), geometry=geom, crs=TARGET_ANALYSIS_CRS)
gdf_ll = gdf.to_crs(TARGET_WEB_CRS)

```

```

master_3d["lon"] = gdf_ll.geometry.x
master_3d["lat"] = gdf_ll.geometry.y

# -----
# depth convention
# keep z_native original, but also provide positive-down depth
# -----
master_3d["depth_m"] = -master_3d["z_native"]

# final column order
master_3d = master_3d[
    ["x_3338", "y_3338", "z_native", "lon", "lat", "surface_name",
     ↪ "source_file", "depth_m"]
].copy()

print(master_3d.shape)
display(master_3d.head())

print(master_3d["surface_name"].value_counts(dropna=False))
print(master_3d.groupby("surface_name")["z_native"].agg(["min", "max", "mean"]))

```

```

Loaded NStopo: (208, 435)
Loaded NSLCU: (208, 435)
Loaded NSshublik: (208, 435)
Loaded NSbasement: (208, 435)
Loaded NStopo-LCU: (208, 435)
Loaded NSLCU-Shublik: (208, 435)
Loaded NSshublik-basement: (208, 435)
Loaded NStopo-basement: (208, 435)
(723840, 8)

```

	x_3338	y_3338	z_native	lon	lat	surface_name \
0	-630000.0	2621000.0	-67.0	-172.658577	72.824670	NStopo
1	-627000.0	2621000.0	-75.0	-172.574298	72.832387	NStopo
2	-624000.0	2621000.0	-82.0	-172.489958	72.840071	NStopo
3	-621000.0	2621000.0	-81.0	-172.405556	72.847722	NStopo
4	-618000.0	2621000.0	-74.0	-172.321093	72.855338	NStopo

	source_file	depth_m
0	NStopo.XYZ	67.0
1	NStopo.XYZ	75.0
2	NStopo.XYZ	82.0
3	NStopo.XYZ	81.0
4	NStopo.XYZ	74.0

surface_name	
NStopo	90480
NSLCU	90480

NSshublik	90480
NSbasement	90480
NStopo-LCU	90480
NSLCU-Shublik	90480
NSshublik-basement	90480
NStopo-basement	90480

Name: count, dtype: int64

	min	max	mean
surface_name			
NSLCU	-16691.0	2084.0	-5244.435345
NSLCU-Shublik	0.0	4283.0	201.815053
NSbasement	-16691.0	2084.0	-6011.202951
NSshublik	-16691.0	2084.0	-5446.249050
NSshublik-basement	0.0	7902.0	564.952796
NStopo	-3801.0	2105.0	-531.923187
NStopo-LCU	0.0	16613.0	4701.515539
NStopo-basement	0.0	16613.0	5468.285566

```
[16]: master_3d.to_parquet("north_slope_master_3d_surfaces.parquet", index=False)
print("Saved: north_slope_master_3d_surfaces.parquet")
```

Saved: north_slope_master_3d_surfaces.parquet

```
[17]: import plotly.graph_objects as go
import pandas as pd

# -----
# choose main surfaces only
# -----
main_surfaces = ["NStopo", "NSLCU", "NSshublik", "NSbasement"]

surface_settings = {
    "NStopo": {"color": "#4daf4a", "size": 2, "opacity": 0.35},
    "NSLCU": {"color": "#377eb8", "size": 2, "opacity": 0.35},
    "NSshublik": {"color": "#ff7f00", "size": 2, "opacity": 0.35},
    "NSbasement": {"color": "#984ea3", "size": 2, "opacity": 0.35},
}

fig = go.Figure()

# -----
# add 3D surfaces as point clouds
# -----
for name in main_surfaces:
    df = master_3d[master_3d["surface_name"] == name].copy()

    fig.add_trace(go.Scatter3d(
        x=df["x_3338"],
```

```

y=df["y_3338"],
z=df["depth_m"],
mode="markers",
name=name,
marker=dict(
    size=surface_settings[name]["size"],
    color=surface_settings[name]["color"],
    opacity=surface_settings[name]["opacity"]
),
hovertemplate=(
    f"<b>{name}</b><br>"
    "X: %{x}<br>"
    "Y: %{y}<br>"
    "Depth: %{z}<extra></extra>"
)
))

# -----
# add wells
# -----
wells_df = master_2d[master_2d["layer_name"] == "wells"].copy()

fig.add_trace(go.Scatter3d(
    x=wells_df["x_3338"],
    y=wells_df["y_3338"],
    z=[0] * len(wells_df),    # wells shown at surface for now
    mode="markers",
    name="Wells",
    marker=dict(
        size=3,
        color="black",
        opacity=0.9,
        symbol="circle"
    ),
    hovertemplate=(
        "<b>Well</b><br>"
        "X: %{x}<br>"
        "Y: %{y}<br>"
        "Z: %{z}<extra></extra>"
    )
))

fig.update_layout(
    title="North Slope Structural Surfaces + Wells",
    scene=dict(
        xaxis_title="Projected X (EPSG:3338)",
        yaxis_title="Projected Y (EPSG:3338)",

```

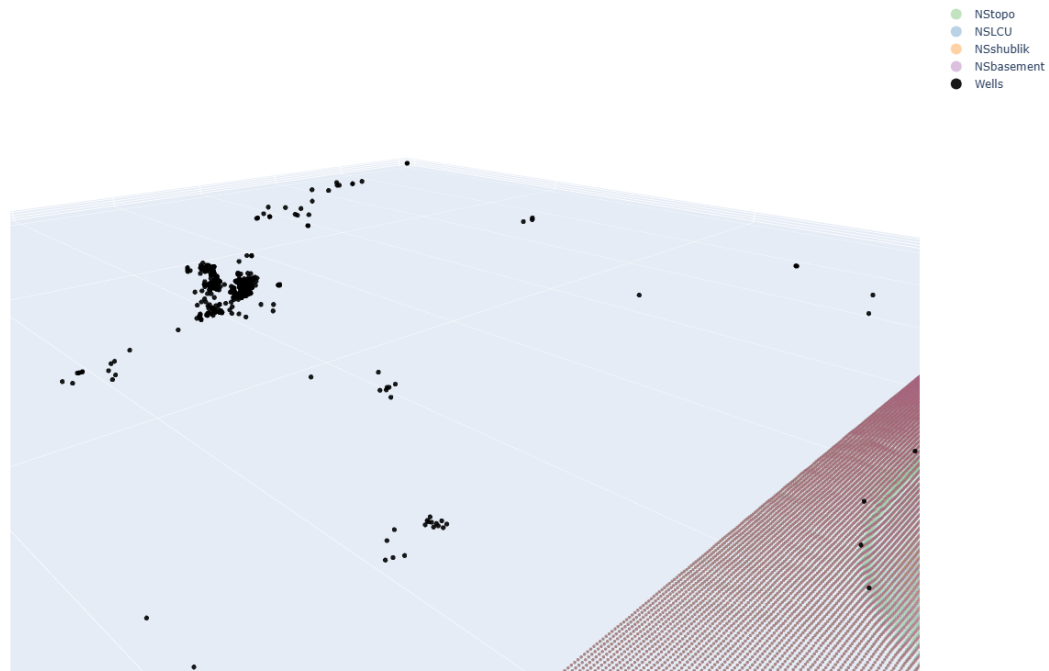
```

        zaxis_title="Depth (m, positive downward)",
        zaxis=dict(autorange="reversed"),
        aspectmode="data",
        camera=dict(
            eye=dict(x=1.5, y=1.5, z=0.8)
        ),
    ),
    height=900,
    width=1300,
    legend=dict(itemsizing="constant")
)

fig.show()

```

North Slope Structural Surfaces + Wells



```

[18]: seis3d_df = master_2d[master_2d["layer_name"] == "seismic_3d_inventory"].copy()

fig.add_trace(go.Scatter3d(
    x=seis3d_df["x_3338"],
    y=seis3d_df["y_3338"],
    z=[0] * len(seis3d_df),

```

```

mode="markers",
name="3D Seismic Inventory",
marker=dict(
    size=1,
    color="red",
    opacity=0.15
),
hoverinfo="skip"
))

```

```

[21]: from pathlib import Path
import subprocess
import sys

# folder where your notebooks are
BASE_DIR = Path.cwd()

# pick the notebooks you want
notebooks = [
    "01_base_map_reorganized.ipynb",
    "01_parquet_prep_cleaned.ipynb",
    "02_geologic_framework.ipynb",
    "Depth_grid_processing.ipynb",
    "North Slope Data Layer Map.ipynb",
    "Master.ipynb",
]

export_dir = BASE_DIR / "notebook_exports"
export_dir.mkdir(exist_ok=True)

for nb in notebooks:
    nb_path = BASE_DIR / nb
    if not nb_path.exists():
        print(f"SKIP: {nb} not found")
        continue

    print(f"Exporting {nb} -> HTML")
    cmd = [
        sys.executable,
        "-m",
        "jupyter",
        "nbconvert",
        "--to",
        "html",
        str(nb_path),
        "--output-dir",
        str(export_dir),
    ]

```

```

]
result = subprocess.run(cmd, capture_output=True, text=True)

if result.returncode == 0:
    print(f"DONE: {nb}")
else:
    print(f"FAILED: {nb}")
    print(result.stderr)

print(f"\nFinished. Exported files are in:\n{export_dir}")

```

```

Exporting 01_base_map_reorganized.ipynb -> HTML
DONE: 01_base_map_reorganized.ipynb
Exporting 01_parquet_prep_cleaned.ipynb -> HTML
DONE: 01_parquet_prep_cleaned.ipynb
Exporting 02_geologic_framework.ipynb -> HTML
DONE: 02_geologic_framework.ipynb
Exporting Depth_grid_processing.ipynb -> HTML
DONE: Depth_grid_processing.ipynb
Exporting North Slope Data Layer Map.ipynb -> HTML
DONE: North Slope Data Layer Map.ipynb
Exporting Master.ipynb -> HTML
DONE: Master.ipynb

```

```

Finished. Exported files are in:
/home/jovyan/notebooks/notebook_exports

```

```

[ ]: from pathlib import Path
import subprocess
import sys

BASE_DIR = Path.cwd()
export_dir = BASE_DIR / "notebook_exports_pdf"
export_dir.mkdir(exist_ok=True)

notebooks = [
    "01_base_map_reorganized.ipynb",
    "01_parquet_prep_cleaned.ipynb",
    "02_geologic_framework.ipynb",
    "Depth_grid_processing.ipynb",
    "North Slope Data Layer Map.ipynb",
    "Master.ipynb",
]

for nb in notebooks:
    nb_path = BASE_DIR / nb
    if not nb_path.exists():

```

```

    print(f"SKIP: {nb} not found")
    continue

print(f"Exporting {nb} -> PDF")
cmd = [
    sys.executable,
    "-m",
    "jupyter",
    "nbconvert",
    "--to",
    "pdf",
    str(nb_path),
    "--output-dir",
    str(export_dir),
]
result = subprocess.run(cmd, capture_output=True, text=True)

if result.returncode == 0:
    print(f"DONE: {nb}")
else:
    print(f"FAILED: {nb}")
    print(result.stderr)

print(f"\nFinished. PDF files are in:\n{export_dir}")

```

```

Exporting 01_base_map_reorganized.ipynb -> PDF
DONE: 01_base_map_reorganized.ipynb
Exporting 01_parquet_prep_cleaned.ipynb -> PDF
DONE: 01_parquet_prep_cleaned.ipynb
Exporting 02_geologic_framework.ipynb -> PDF
DONE: 02_geologic_framework.ipynb
Exporting Depth_grid_processing.ipynb -> PDF
DONE: Depth_grid_processing.ipynb
Exporting North Slope Data Layer Map.ipynb -> PDF
DONE: North Slope Data Layer Map.ipynb
Exporting Master.ipynb -> PDF

```

[]:

[]:

[]:

[]:

[]:

[]: